SHARC-Drive: Hardware-Aware Simulation for Real-Time Autonomous Driving

Author 1*, Author 2†, Author 3*
*Affiliation 1 †Affiliation 2
{email1, email2}@placeholder.edu

***Abstract*—In autonomous driving, control algorithms ranging from PID to nonlinear model predictive control (NMPC) must be executed within a strict sample period on embedded processors whose computational resources vary widely. The computation time depends on the microarchitecture—clock speed, cache size, pipeline depth—and interacts with the controller's algorithmic complexity in ways that are difficult to predict analytically. This paper presents a co-simulation framework coupling *SHARC* (Simulator for Hardware Architecture and Real-time Control) with the *CARLA* driving simulator, enabling systematic evaluation of how hardware and controller design choices jointly affect closed-loop safety.**

***Index Terms*—Hardware–software co-design, autonomous driving, model predictive control, computational delay, microarchitecture simulation.**

I. Introduction

An autonomous vehicle's control loop follows a simple pattern: sense, compute, actuate. The critical assumption in most control designs is that computation is instantaneous. On a fast processor running a PID controller, this assumption is reasonable—the computation completes in microseconds. For a nonlinear model predictive control (MPC) algorithm on constrained hardware, however, the computation may approach or exceed the sample period T. When that occurs, the controller misses its deadline: no fresh command is delivered, and the vehicle continues executing the previous one.

This coupling between processor capability and control performance is well understood in principle [**?**], [**?**], yet the two domains are typically designed in isolation. Control engineers assume instantaneous computation; hardware engineers optimize for generic benchmarks. Testing on physical prototypes discovers timing violations late and expensively [**?**].

A simulation framework that evaluates the complete loop, plant physics, control algorithm, *and* processor microarchitecture, in a single experiment is needed. This paper presents such a framework by combining two simulators:

- **SHARC** [**?**] (Simulator for Hardware Architecture and Real-time Control) executes a controller binary on a parameterized processor model via the Scarab cycle-accurate simulator [**?**], producing the computation time τ_k at every sample step.
- **CARLA** [**?**], an open-source driving simulator built on Unreal Engine, provides realistic vehicle dynamics, urban road networks, and traffic.

SHARC is designed around an *abstract dynamics interface*: any function mapping the current state and control to the next state can serve as the plant. We exploit this by replacing the default ODE integrator with CARLA's physics engine, thereby obtaining high-fidelity vehicle dynamics without modifying SHARC's core architecture.

Contributions. **[TODO:]**

II. Modeling Framework

We develop a general model of a sampled-data system in which the controller's computation time depends on the state, algorithm, and hardware. The model is independent of any particular plant; CARLA enters in Section III as one instantiation.

A. Plant and Discretization

Consider a continuous-time plant

$$\dot{x} = \tilde{f}(t, x, u, w), \qquad y = h(x), \tag{1}$$

with state $x \in \mathbb{R}^n$, control input $u \in \mathbb{R}^m$, exogenous input $w \in \mathbb{R}^p$, and measured output $y \in \mathbb{R}^q$. A digital controller samples the output at times $t_k := kT$, $k \in \mathbb{N}$, with sample period $T > 0$. Under zero-order hold, the control is constant over each interval $[t_k, t_{k+1})$, yielding the discrete-time system

$$x_{k+1} = f(x_k, u_k, w_k), \tag{2}$$

where f encapsulates the continuous dynamics integrated over one sample period under constant u_k and $w_k := w(t_k)$.

Remark 1. *The map f in (2) need not have a closed-form expression. It may be evaluated by a numerical integrator, a lookup table, or, as in our framework, an external physics engine. This generality is central to the design of SHARC and enables the CARLA integration described in Section III-B.*

B. Computation Delay and Zero-Order Hold

At each sample time t_k, the controller reads $y_k = h(x_k)$ and begins computing a new input $\tilde{u}_{k+1}$. The computation requires $\tau_k > 0$ seconds. The computed input is applied at the *next* sample instant if and only if the computation finishes within one period; otherwise, the previous input is held.

Definition 1 (Zero-Order Hold Under Delay)**.** *The input applied at sample $k+1$ is*

$$u_{k+1} = \begin{cases} \tilde{u}_{k+1} & \text{if } \tau_k < T, \\ u_k & \text{if } \tau_k \geq T. \end{cases} \tag{3}$$

Definition 2 (Deadline Miss)**.** *At time step k, a* deadline miss *occurs if $\tau_k \geq T$. The* miss rate *over a horizon of K steps is*

$$\rho := \frac{1}{K} \sum_{k=0}^{K-1} \mathbf{1}[\tau_k \geq T].$$

C. Hardware-Dependent Computation Time

The computation time depends on the algorithm g, the current measurement y_k, and the hardware configuration $\mathcal{H} \in \mathbb{R}_+^{n_h}$ (clock frequency, cache sizes, pipeline parameters):

$$\tau_k = \mathcal{T}(g, y_k, \mathcal{H}). \tag{4}$$

For a PID controller, $\mathcal{T}$ is effectively constant: the computation is a fixed sequence of multiplications and additions. For MPC, $\mathcal{T}$ depends on y_k through the optimization landscape, states far from the reference require more solver iterations, and on $\mathcal{H}$ through instruction throughput.

Evaluating $\mathcal{T}$ analytically is intractable for nontrivial controllers on realistic processors. SHARC evaluates it *empirically* by executing the controller binary inside the Scarab cycle-accurate simulator for a given hardware configuration.

D. Closed-Loop Dynamics and Cascading Misses

Substituting (3) into (2), the closed-loop dynamics become

$$x_{k+1} = f\big(x_k,\ \mu(x_{0:k},\ \tau_{0:k-1}),\ w_k\big), \tag{5}$$

where μ is the effective input policy determined by the ZOH rule. Because τ_k depends on x_k via (4), a feedback loop emerges: the state influences the computation time, which determines the applied input, which drives the next state.

Remark 2 (Cascading deadline misses)**.** *For controllers whose computation time grows with the tracking error or proximity to constraints, as is typical for iterative optimization-based methods such as MPC, deadline misses can cascade. When a miss occurs, the stale input drives the state further from the reference. The larger error, in turn, requires more solver iterations at the next step, increasing τ_{k+1} and making a subsequent miss more likely. This positive feedback loop can rapidly destabilize the closed loop: a single miss triggers growing errors and ever-longer computation times, producing a sharp* performance cliff*. By contrast, controllers with state-independent computation time (e.g., PID) are immune to this cascade, since τ_k does not depend on the tracking error.*

III. Co-Simulation Framework

A. SHARC

SHARC [**?**] is a framework for evaluating closed-loop control under realistic computational delays. Its architecture is organized around two user-supplied modules and one microarchitecture simulator:

Dynamics. The user provides a Python class that implements the state transition f in (2). The class exposes two methods: `initialize()`, which sets up the plant state, and `step(x, u, w)`, which advances the dynamics by one sample period and returns x_{k+1}. Because the dynamics are defined in Python, users can implement analytical models (e.g., an ODE integrator), data-driven surrogates, or, as in this work, a bridge to an external simulator.

Controller. The controller is compiled to a native binary that reads the measurement (t_k, y_k, w_k) and generates the control input $\tilde{u}_{k+1}$. Any control algorithm that can be expressed in C or C++ is supported by SHARC.

Scarab. **[TODO: Shengmin: Check and verify this explanation and make it more accurate.]** The Scarab cycle-accurate simulator [**?**] executes the controller binary on a user-defined parameterized processor model, producing the computation time τ_k in simulated clock cycles. Scarab models instruction caches, data caches, branch predictors, out-of-order pipelines, and memory hierarchy. The hardware configuration $\mathcal{H}$ is specified via a parameter file; sweeping the design space (e.g., varying clock frequency or cache size) requires only editing the corresponding parameter.

SHARC provides two execution modes. In *execution-driven* (serial) mode, Scarab simulates the controller instruction-by-instruction during the control loop, producing τ_k at each step. In *trace-driven* (parallel) mode, the controller runs natively under DynamoRIO instrumentation [**?**] to capture memory-access traces, which Scarab replays offline; multiple steps are processed in parallel for faster processing. When a deadline miss is detected, SHARC re-simulates from the miss point with the corrected state under (3) as discussed in the SHARC paper [**?**].

B. CARLA Integration

CARLA [**?**] is an open-source driving simulator with realistic vehicle dynamics, urban road networks, and traffic participants. We integrate CARLA into SHARC by implementing a Python dynamics class that connects to CARLA's synchronous API. At initialization, the class spawns the ego vehicle and NPC traffic, enables synchronous mode with physics step $\Delta t = T$, and seeds the random number generators for reproducibility. At each step, the class applies the control input to CARLA, ticks the world, and reads back the new state. Because $\Delta t = T$, each CARLA tick corresponds to exactly one discrete-time step in (2). From SHARC's perspective, CARLA is simply a black-box instantiation of the abstract f. **[TODO: Yash & Malavikha: explain our determinism framework, explain why we simulate from the beginning for each batch.]**

IV. Controller Formulations

We evaluate two controllers with qualitatively different computational profiles: a PID controller with constant per-step cost and a nonlinear MPC with obstacle-avoidance constraints whose per-step cost depends on the state and the traffic environment.

A. State, Control, and Exogenous Inputs

The ego vehicle state is

$$x_k := (p_x,\ p_y,\ \psi,\ v)^\top \in \mathbb{R}^4, \tag{6}$$

where (p_x, p_y) is the global position, ψ the heading (yaw), and v the longitudinal speed.

PID control input. The PID controller outputs CARLA-native commands

$$u_k^{\mathrm{PID}} := (\alpha,\ \delta,\ \beta)^\top \in [0,1] \times [-1,1] \times [0,1], \tag{7}$$

with throttle α, steering δ, and brake β.

MPC control input. The MPC controller operates on physical quantities

$$u_k^{\mathrm{MPC}} := (a,\ \delta)^\top \in [a_{\min}, a_{\max}] \times [\delta_{\min}, \delta_{\max}], \tag{8}$$

where a is the longitudinal acceleration ($\mathrm{m/s^2}$) and δ the front-wheel steering angle (rad). The dynamics layer maps (a, δ) to CARLA's throttle/brake/steer commands before actuation.

Exogenous input. At each step, the dynamics module queries a set of N_w reference waypoints $\mathcal{W}_k := \{w_k^{(i)}\}_{i=1}^{N_w}$ from CARLA's HD map and detects up to N_o nearby obstacles, yielding the exogenous vector

$$w_k := \big(\underbrace{w_x^{(1)}, w_y^{(1)}, \ldots, w_x^{(N_w)}, w_y^{(N_w)}}_{2N_w},\ \underbrace{o_1, \ldots, o_{N_o}}_{5N_o}\big)^\top \in \mathbb{R}^{2N_w+5N_o} \tag{9}$$

where each obstacle entry $o_i := (o_x^i, o_y^i, \dot{o}_x^i, \dot{o}_y^i, r_i)$ contains the position, velocity, and bounding-circle radius. Empty obstacle slots are marked with a sentinel value ($o_x \geq 5{\times}10^5$) so that the solver treats them as vacuously satisfied.

B. PID Controller

The PID controller decomposes vehicle control into two independent SISO loops.

Longitudinal. Track a constant reference speed v^*:

$$\sigma_k = K_P^\ell\, e_k^v + K_I^\ell \sum_{j=0}^{k} e_j^v\, T + K_D^\ell\, \frac{e_k^v - e_{k-1}^v}{T}, \tag{10}$$

where $e_k^v := v^* - v_k$. Throttle and brake are $\alpha_k = \mathrm{clip}(\sigma_k,\ 0,\ 1)$, $\beta_k = \mathrm{clip}(-\sigma_k,\ 0,\ 1)$.

Lateral. Steer toward the next waypoint. The heading error is $e_k^\psi := \mathrm{atan2}(w_y^{(1)} - p_y,\ w_x^{(1)} - p_x) - \psi_k$, and the steering command is

$$\delta_k = \mathrm{clip}\big(K_P^r\, e_k^\psi + K_I^r \textstyle\sum_j e_j^\psi\, T + K_D^r\, (e_k^\psi - e_{k-1}^\psi)/T,\ \pm 1\big). \tag{11}$$

PID requires a fixed number of arithmetic operations per step, so $\tau_k \approx \mathrm{const}$.

C. Obstacle-Aware Model Predictive Controller

The MPC controller receives the reference waypoints $\mathcal{W}_k$ and obstacle states from (9) and solves a constrained nonlinear program at every sample step. The formulation is illustrated in Fig. 1.

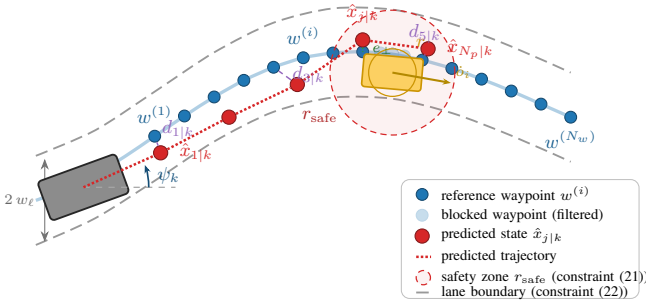

Fig. 1. Obstacle-aware MPC formulation. The ego vehicle (gray) tracks reference waypoints $\{w^{(i)}\}$ (blue dots) along the lane center while following a leading NPC (orange). Each obstacle is enclosed by a bounding circle of radius r_i and a safety zone of radius $r_{\mathrm{safe}} = r_{\mathrm{ego}} + r_i + m$ (red dashed). The reference path is *truncated* at the first blocked waypoint: all subsequent waypoints collapse to the last safe position (faded dots), so MPC sees no path beyond the obstacle. Simultaneously, the reference speed adapts to the leading vehicle's speed (15). Together, these mechanisms cause MPC to decelerate and follow rather than swerve. Lane boundaries (gray dashed) enforce (22).

1) Prediction Model: The MPC uses a kinematic bicycle model for internal prediction. Given the current predicted state $\hat{x}_{j|k} = (p_x, p_y, \psi, v)^\top$ and control $u_{j|k} = (a, \delta)^\top$, the next state is obtained by forward-Euler integration:

$$\hat{x}_{j+1|k} = \hat{x}_{j|k} + T \begin{bmatrix} v_{j|k} \cos\psi_{j|k} \\ v_{j|k} \sin\psi_{j|k} \\ (v_{j|k}/L)\ \tan\delta_{j|k} \\ a_{j|k} \end{bmatrix}, \tag{12}$$

where L is the wheelbase.

2) Reference Waypoint Truncation: Before each optimization, the dynamics layer constructs the waypoint set $\mathcal{W}_k$ by querying CARLA's road network ahead of the ego vehicle. When an obstacle blocks the path, the reference is *truncated*: the first waypoint that falls inside any obstacle's exclusion zone triggers replacement of *all* subsequent waypoints with the last unblocked one. Formally, define the *truncation index*

$$i^\star := \min\big\{\, i \mid \exists\, \ell :\ \big\|w_k^{(i)} - o_k^{(\ell)}\big\| < r_{\mathrm{safe}}^{(\ell)} \big\}, \tag{13}$$

and set

$$\tilde{w}_k^{(i)} := \begin{cases} w_k^{(i)} & \text{if } i < i^\star, \\ w_k^{(i^\star - 1)} & \text{if } i \geq i^\star. \end{cases} \tag{14}$$

Because every waypoint beyond the obstacle collapses to a single point, MPC sees no reference path past the blocked region and has no incentive to swerve around the obstacle. Combined with the adaptive reference speed (below), this causes the controller to decelerate and follow the leading vehicle.

3) Adaptive Reference Speed: When a leading vehicle is detected, the reference speed is adapted to prevent the optimizer from requesting acceleration toward an unreachable cruise speed. Let $\mathcal{L}_k := \{\ell \mid (o_k^{(\ell)} - p_k)^\top \hat{e}_\psi > 0\}$ denote the set of obstacles ahead of the ego, where $\hat{e}_\psi := (\cos\psi_k, \sin\psi_k)^\top$. The effective reference speed is

$$v_k^* := \min\Big(v^*,\ \min_{\ell \in \mathcal{L}_k} \max\big(\dot{o}_k^{(\ell)\top} \hat{e}_\psi,\ 0\big)\Big), \tag{15}$$

where the inner product $\dot{o}_k^{(\ell)\top}\hat{e}_\psi$ projects the obstacle's velocity onto the ego heading, and the $\max(\cdot,0)$ ensures that oncoming traffic does not produce a negative target. When no obstacle is ahead ($\mathcal{L}_k=\emptyset$), the nominal speed v^* is used.

4) Obstacle Prediction: Each detected obstacle is propagated forward with a constant-velocity model over the prediction horizon:

$$\hat{o}_j^{(\ell)} := o_k^{(\ell)} + j\,T\,\dot{o}_k^{(\ell)}, \qquad j=1,\dots,N_p, \tag{16}$$

where $o_k^{(\ell)}=(o_x^\ell,o_y^\ell)^\top$ and $\dot{o}_k^{(\ell)}=(\dot{o}_x^\ell,\dot{o}_y^\ell)^\top$ are the position and velocity of obstacle ℓ at time t_k.

5) Cost Function: The objective penalizes waypoint-tracking error, speed deviation, control effort, and input jerk:

$$J = J_{\text{track}} + J_{\text{speed}} + J_{\text{effort}} + J_{\text{jerk}}. \tag{17}$$

Waypoint tracking. The distance from each predicted position to the closest (filtered) reference waypoint is

$$d_{j|k} := \min_{i\in\{1,\dots,N_w\}} \big\|(p_{x,j|k},\,p_{y,j|k}) - \tilde{w}_k^{(i)}\big\|, \tag{18}$$

and the tracking cost is $J_{\text{track}} := \sum_{j=1}^{N_p}\gamma^j\,q_\ell\,d_{j|k}^2$.

Speed tracking. $J_{\text{speed}} := \sum_{j=1}^{N_p}\gamma^j\,q_v\,(v_{j|k}-v_k^*)^2$, where v_k^* is the adaptive reference speed (15).

Control effort. $J_{\text{effort}} := \sum_{j=0}^{N_c-1}\gamma^j\big(r_a\,a_{j|k}^2 + r_\delta\,\delta_{j|k}^2\big)$.

Input jerk. Penalizing successive control differences discourages abrupt actuator changes:

$$J_{\text{jerk}} := \sum_{j=0}^{N_c-1}\big(r_{\Delta a}\,\Delta a_{j|k}^2 + r_{\Delta\delta}\,\Delta\delta_{j|k}^2\big), \tag{19}$$

where $\Delta a_{0|k} := a_{0|k} - a_{k-1}$ and $\Delta a_{j|k} := a_{j|k} - a_{j-1|k}$ for $j\geq 1$ (analogously for δ), and a_{k-1} is the acceleration applied at the previous time step. The discount factor $\gamma\in(0,1]$ down-weights distant prediction steps.

6) Hard Safety Constraints: Safety is enforced through inequality constraints rather than penalty terms, ensuring that no feasible trajectory penetrates an obstacle or leaves the lane.

Collision avoidance. Each obstacle ℓ and the ego vehicle are enclosed by bounding circles of radii r_ℓ and r_{ego}, respectively. Defining the safe radius as

$$r_{\text{safe}}^{(\ell)} := r_{\text{ego}} + r_\ell + m, \tag{20}$$

with safety margin $m>0$, the collision-avoidance constraint at prediction step j is

$$\big\|(p_{x,j|k},\,p_{y,j|k}) - \hat{o}_j^{(\ell)}\big\|^2 \geq \big(r_{\text{safe}}^{(\ell)}\big)^2, \quad \begin{matrix}\ell=1,\dots,N_o,\\ j=1,\dots,N_p.\end{matrix} \tag{21}$$

Lane keeping. Let $e_{\perp,j|k}$ denote the signed perpendicular distance from the predicted position $(p_{x,j|k},p_{y,j|k})$ to the closest segment of the reference path, computed by projecting onto the polyline $(\tilde{w}^{(1)},\dots,\tilde{w}^{(N_w)})$:

$$e_{\perp,j|k}^2 \leq w_\ell^2, \qquad j=1,\dots,N_p, \tag{22}$$

where w_ℓ is half the lane width. This enforces that the ego vehicle remains within the drivable corridor at every predicted step.

Constraints (21) and (22) yield $N_p(N_o+1)$ scalar inequalities per optimization.

7) Optimization Problem: Collecting the above, the MPC solves at each t_k:

$$\min_{u_{0|k},\dots,u_{N_c-1|k}} \quad J \tag{23a}$$

$$\text{s.t.} \quad \hat{x}_{j+1|k} = F(\hat{x}_{j|k},\,u_{j|k}), \tag{23b}$$

$$\hat{x}_{0|k} = x_k, \tag{23c}$$

$$u_{j|k}\in\mathcal{U}, \qquad j=0,\dots,N_c-1, \tag{23d}$$

$$(21),\ (22), \qquad j=1,\dots,N_p, \tag{23e}$$

with prediction horizon N_p, control horizon $N_c\leq N_p$, and input constraint set $\mathcal{U} := [a_{\min},a_{\max}]\times[\delta_{\min},\delta_{\max}]$. Only the first input $u_{0|k}$ is applied (receding horizon); the result becomes $\tilde{u}_{k+1}$ in (3).

8) Infeasibility Handling: When traffic configurations render (23) infeasible (e.g., a stopped vehicle fills the lane within r_{safe}), the controller falls back to an emergency policy:

$$u_k^{\text{emerg}} := \begin{cases}(a_{\min},\,0) & \text{if } |v_k|\geq v_{\text{thr}},\\ (0,\,0) & \text{if } |v_k|<v_{\text{thr}},\end{cases} \tag{24}$$

i.e., maximum braking until the vehicle is nearly stopped, then holding. A *sticky-hold* flag prevents re-acceleration until all obstacles are farther than $2\,r_{\text{safe}}$, ensuring that the ego vehicle does not oscillate between braking and accelerating in tight traffic.

Remark 3 (State-dependent computation time)**.** *The number of solver iterations, and hence τ_k, depends on the current state and the obstacle configuration. Near the reference with no obstacles, the optimization is well-conditioned; when obstacles activate constraints* (21)*, the feasible set shrinks and the solver requires more iterations. After a deadline miss, the stale input drives the state further from the reference, compounding difficulty. This is the mechanism underlying the cascading misses described in Remark 2.*

Remark 4 (Computational complexity ordering)**.** *PID requires $O(1)$ arithmetic operations per step. The obstacle-aware MPC iteratively solves a nonconvex NLP with $2N_c$ decision variables and $N_p(N_o+1)$ inequality constraints, whose cost grows with N_p, N_c, and N_o. The ordering $\tau_k^{\text{PID}} \ll \tau_k^{\text{MPC}}$ is central to our hardware sensitivity analysis.*

V. Experimental Setup

A. Driving Scenarios

Table I summarizes the scenarios drawn from CARLA's built-in maps and town configurations. Scenarios are ordered by *safety criticality*: the higher the row, the less margin is available for delayed actuation. **[TODO: Fill this table correctly]**

B. Hardware Configurations

We evaluate each controller on a set of processor configurations by varying the clock frequency and L1 data cache size while holding all other microarchitectural parameters (pipeline

TABLE I
DRIVING SCENARIOS USED FOR EVALUATION.

Scenario	Map	Key challenge	T (s)
Straight-road cruising	Town04	Speed tracking	0.25
Highway lane following	Town04	Lateral precision	0.25
Adaptive cruise control	Town04	Lead-car gap	0.20
Intersection crossing	Town03	Timing, collision av.	0.15
Emergency braking	Town01	Min. stopping dist.	0.10
High-speed race lap	Town04	Stability at speed	0.10

TABLE II
PROCESSOR CONFIGURATIONS EVALUATED IN ALL EXPERIMENTS. EACH ROW DEFINES ONE SCARAB PARAMETER SET.

Config	f_{clk} (MHz)	S_{L1} (kB)	Description
A	2000	64	Fast baseline
B	1000	64	Halved frequency
C	500	64	Quarter frequency
D	250	64	Eighth frequency
E	2000	8	Small cache
F	2000	128	Large cache
G	500	8	Constrained

width, branch predictor, memory latency) at a fixed baseline. Table II summarizes the configurations.

C. Performance Metrics

Let $\{x_k\}_{k=0}^{K}$ be the simulated trajectory and $\{\bar{x}_k\}_{k=0}^{K}$ the reference.

Position RMSE:

$$\text{RMSE}_{\text{pos}} := \sqrt{\frac{1}{K}\sum_{k=0}^{K-1}\left[(p_x^k - \bar{p}_x^k)^2 + (p_y^k - \bar{p}_y^k)^2\right]}. \quad (25)$$

Speed RMSE: $\text{RMSE}_v := \sqrt{\frac{1}{K}\sum_{k=0}^{K-1}(v_k - v^*)^2}$.

Deadline miss rate ρ (Definition 2).

Mean computation delay: $\bar{\tau} := \frac{1}{K}\sum_{k=0}^{K-1}\tau_k$, with distribution $p(\tau_k)$ visualized as a box plot.

Safety events: number of (a) collisions with the environment, (b) lane-boundary violations ($|e_y| > 1.5\,\text{m}$), and (c) speed-limit exceedances ($v > 1.2\,v^*$) per episode.

VI. RESULTS

We report results for lane following with $T = 0.1\,\text{s}$ and $K = 64$ steps. Table III presents the key metrics for each hardware configuration.

A. Computation Delay Distributions

[TODO: Figure: Box plots of τ_k distributions for PID, MPC across different hardware and scenarios]

B. Tracking Performance vs. Hardware

[TODO: Figure: RMSE_{pos} vs. clock frequency sweep for NLMPC and LMPC on the intersection and emergency-braking scenarios. Expected finding: step-function degradation at a critical clock—the "hardware–performance cliff." PID flat across all frequencies, confirming computational simplicity as robustness.]

TABLE III
EXPERIMENTAL RESULTS: MEAN COMPUTATION DELAY $\bar{\tau}$, DEADLINE MISS RATE ρ, POSITION RMSE, AND SPEED RMSE FOR EACH CONTROLLER AND HARDWARE CONFIGURATION ($T = 0.1\,\text{s}$, $K = 64$ STEPS).

		PID		MPC	
Cfg	Metric	Value	ρ	Value	ρ
A	$\bar{\tau}$ (μs)	14	0.0	—	—
	RMSE_{pos} (m)	—		—	
B	$\bar{\tau}$ (μs)	28	0.0	—	—
	RMSE_{pos} (m)	—		—	
C	$\bar{\tau}$ (μs)	56	0.0	—	—
	RMSE_{pos} (m)	—		—	
D	$\bar{\tau}$ (μs)	112	0.0	—	—
	RMSE_{pos} (m)	—		—	
G	$\bar{\tau}$ (μs)	56	0.0	—	—
	RMSE_{pos} (m)	—		—	

[TODO: Figure: Pareto frontier plots: RMSE_{pos} vs. clock speed for LMPC and NLMPC with varying prediction horizon $N_p \in \{5, 10, 20\}$. Each (N_p, f_{clk}) pair is a design point. Expected finding: smaller N_p reduces computation time, lowers ρ, but increases tracking error; the Pareto frontier identifies optimal (N_p, f_{clk}) tradeoffs for each AV scenario.]

This analysis operationalizes the co-design objective: rather than choosing the largest N_p that a given processor can *nominally* run, the SHARC–CARLA framework reveals the largest N_p the processor can run *within the sample period* of a specific driving scenario, accounting for worst-case instruction-level timing.

C. Cache Sensitivity

[TODO: Cache sensitivity plots]

VII. DISCUSSION

[TODO: This is just a placeholder for now.] The performance cliff. The most significant finding in SHARC-based experiments [**?**] is that control degradation is not gradual. A small reduction in processor capability can flip the system from zero misses to cascading misses (Remark 2). Our CARLA integration extends this observation to a realistic driving scenario with high-fidelity vehicle dynamics and traffic. **MPC horizon as a co-design variable.** The classical MPC design question "how large should N_p be?" is answered in closed-loop terms by our Pareto analysis. On constrained hardware, halving N_p can eliminate all deadline misses while incurring only a modest RMSE increase—a trade that the designer cannot evaluate without SHARC.

Limitations. CARLA's physics engine, while detailed, is not a certified vehicle model. The current study evaluates only two controllers; extending to a broader family of algorithms is future work.

VIII. Conclusion

[TODO:]